

I think are the most important text-processing features. For comprehensive information, you will have to study the Bash Reference Manual. Bash and several commands such as *sed* and *grep* also use regular expressions to perform text processing. ‘Regular expressions’ is a separate topic on its own and I will not cover it either.

History expansion character (!)

This feature is available when typing commands at the shell prompt. It is used to access commands stored in the bash history file.

<i>!n</i>	Execute nth command in bash history
<i>!!</i>	Execute last command (Equivalent to <i>!-1</i>)
<i>!leword</i>	Execute last command <i>beginning</i> with ‘leword’
<i>!?leword?</i>	Execute last command <i>containing</i> ‘leword’
<i>^search^replace</i>	Execute last command after replacing first occurrence of ‘search’ with ‘replace’

You can modify the history search using certain *word designators*, preceded by a colon (:).

<i>!?leword?:0</i>	Execute with 0 th word (usually the command) executable in last command containing ‘leword’
<i>!?leword?:2</i>	Execute with second word of last command containing ‘leword’
<i>!?leword?:\$</i>	Execute with last word in last command containing ‘leword’
<i>!?leword?:2-6</i>	Execute with second word to sixth word in last command containing ‘leword’
<i>!?leword?:-6</i>	Execute with all words up to 6 th word in last command containing ‘leword’ (Equivalent to <i>!?leword?:0-6</i>)
<i>!?leword?:*</i>	Execute with all words of last command (but not the 0 th word) containing ‘leword’ (Equivalent to <i>!?leword?:1-\$</i>)
<i>!?leword?:2*</i>	Execute with the second word to the last word in last command (but not the 0 th word) containing ‘leword’ (Equivalent to <i>!?leword?:2-\$</i>)
<i>!?leword?:2-</i>	Execute with all words from the 2 nd position to last but-one word and not the 0 th word in the command containing ‘leword’

Remember that bash will execute whatever you have retrieved from the history with whatever you have already typed at the prompt.

You can also use any number of *modifiers*, each preceded by a colon (:).

<i>!?leword?:p</i>	Display (but not execute) last command containing ‘leword’
<i>!?leword?:t</i>	Execute with last command containing ‘leword’ after removing all pathnames of last argument (i.e., leave the tail containing the file name)
<i>!?leword?:r</i>	Execute with last command containing ‘leword’ after removing the file extension from the last argument
<i>!?leword?:e</i>	Execute with last command containing ‘leword’ after removing pathname and filename from the last argument (leaving just the extension)
<i>!?leword?:s/search/replace</i>	Execute last command containing ‘leword’ after replacing the first instance of ‘search’ with ‘replace’
<i>!?leword?:as/search/replace</i>	Execute last command containing ‘leword’ after replacing all instances of ‘search’ with ‘replace’

If you omit the search text (‘leword’) and use the history expansion character with the word designators and the modifiers, bash will search the last command. Until you become proficient in using the history expansion character, use the modifier *:p* to display the command before you actually execute it.

Text expansions and substitutions

These features are available at the shell prompt and in shell scripts.

- **Tilde (~):** In your commands, bash will expand instances of ~ with the value of the environmental variable \$HOME, that is, your home directory.
- **? and *:** These are metacharacters. In file descriptors, ? matches any one character while * matches any number of any characters. If they do not match any file names, bash will use their literal values.
- **Brace expansion:** You can use comma-separated text strings within curly brackets to generate combinations of strings with their suffixes and/or prefixes. When I start a new book, I create its folders like this.

```
mkdir -p NewBook/{ebook/images,html/images,image-sources,isbn,pub,ref}
```

This command creates folders like this.

```
NewBook
NewBook/ref
NewBook/pub
NewBook/isbn
```